



Ministry of Higher Education and Scientific Research
Northern Technical University
Technical Engineering College \ Mosul
Computer Technical Engineering



Router AQM QUEUE PIE & FIFO ALGORITHMS

Presented by

Arshad Luqman

Thanoon Younis

Maryam Majid

Supervisor

Dr. Ahmad Faleh

Network Queue Management Simulation Table of Contents

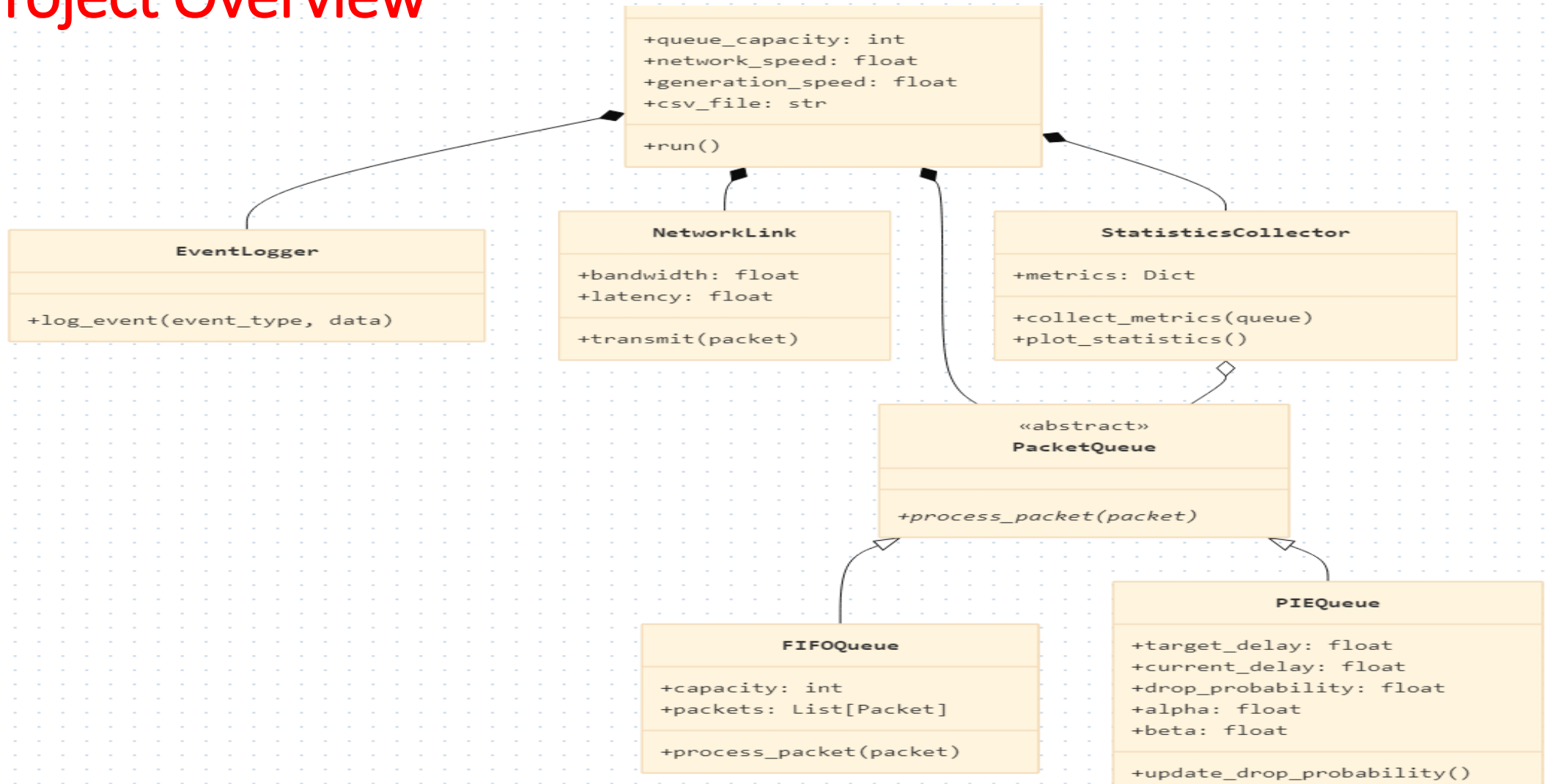
- 1. Project Overview**
- 2. Algorithms - FIFO Algorithm - PIE Algorithm**
- 3. Project Structure**
- 4. Implementation Details 5.**

1. Project Overview

Network queue management is crucial for maintaining efficient packet processing in modern networks. This simulation compares two fundamental approaches:

1. **First-In-First-Out (FIFO):** A basic queuing strategy where packets are processed strictly in arrival order
2. **Proportional Integral controller Enhanced (PIE):** An advanced algorithm using control theory to manage queue delays actively

Project Overview



2. Algorithms

2.1 FIFO Algorithm

The First-In-First-Out algorithm implements a simple queue management strategy where packets are processed strictly in arrival order. Its mathematical model is defined as:

$$Q(t) = \min(Q(t-1) + A(t) - D(t), C)$$

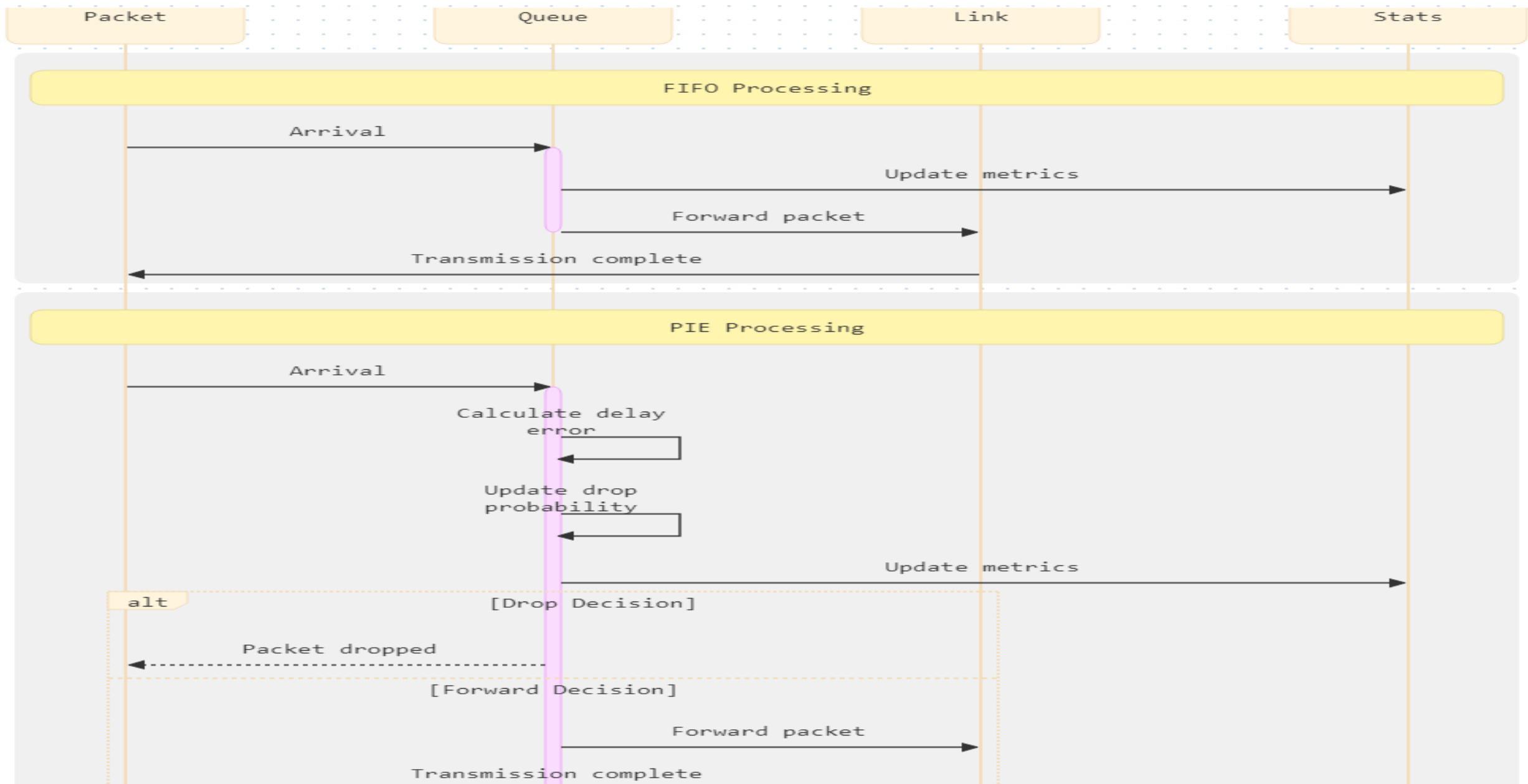
Where:

$Q(t)$: Current queue size

$A(t)$: Arriving packets

$D(t)$: Departing packets

C : Queue capacity



2.2 PIE Algorithm

PIE implements active queue management using control theory principles. Its core components include:

1. Delay Calculation:

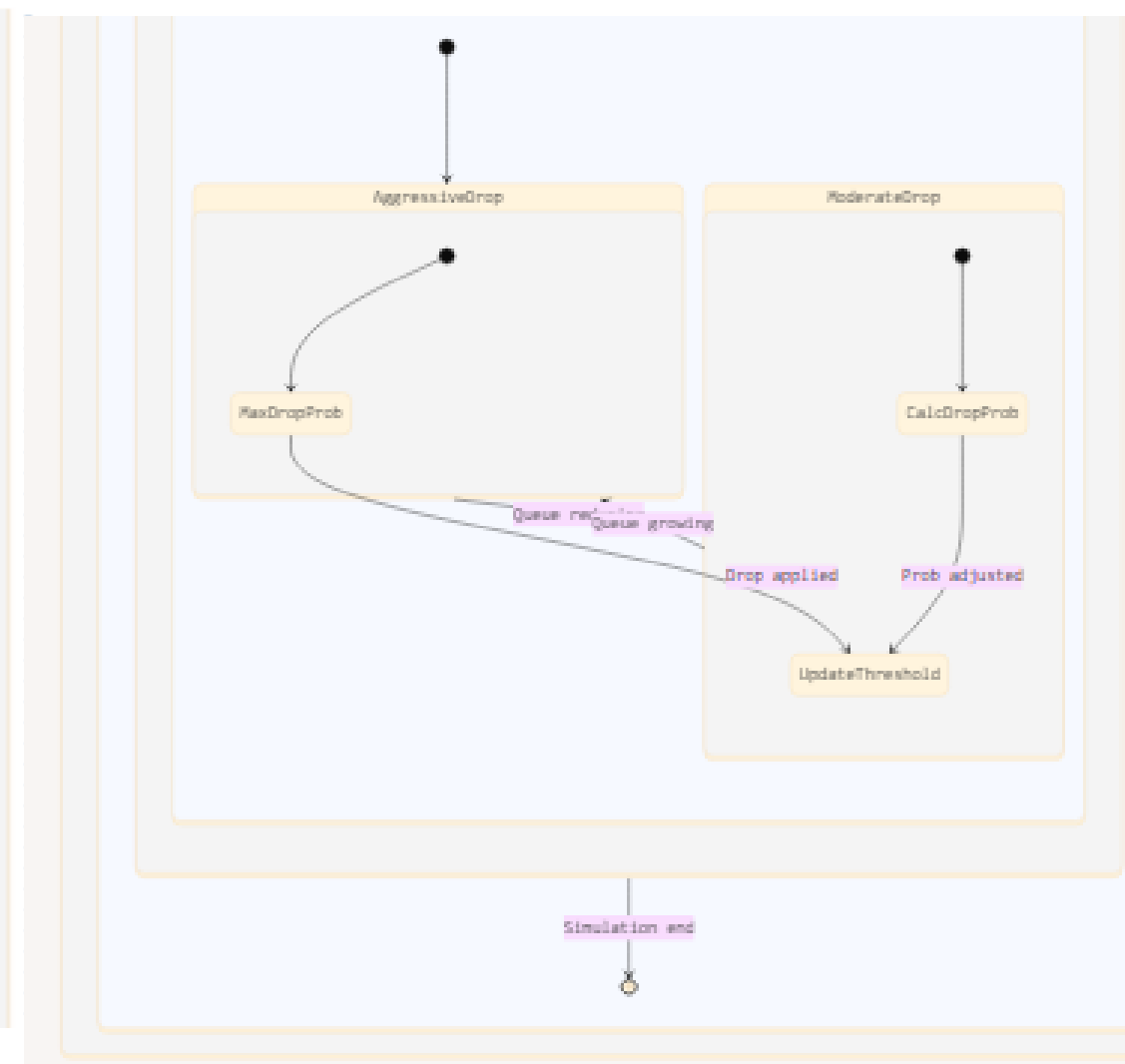
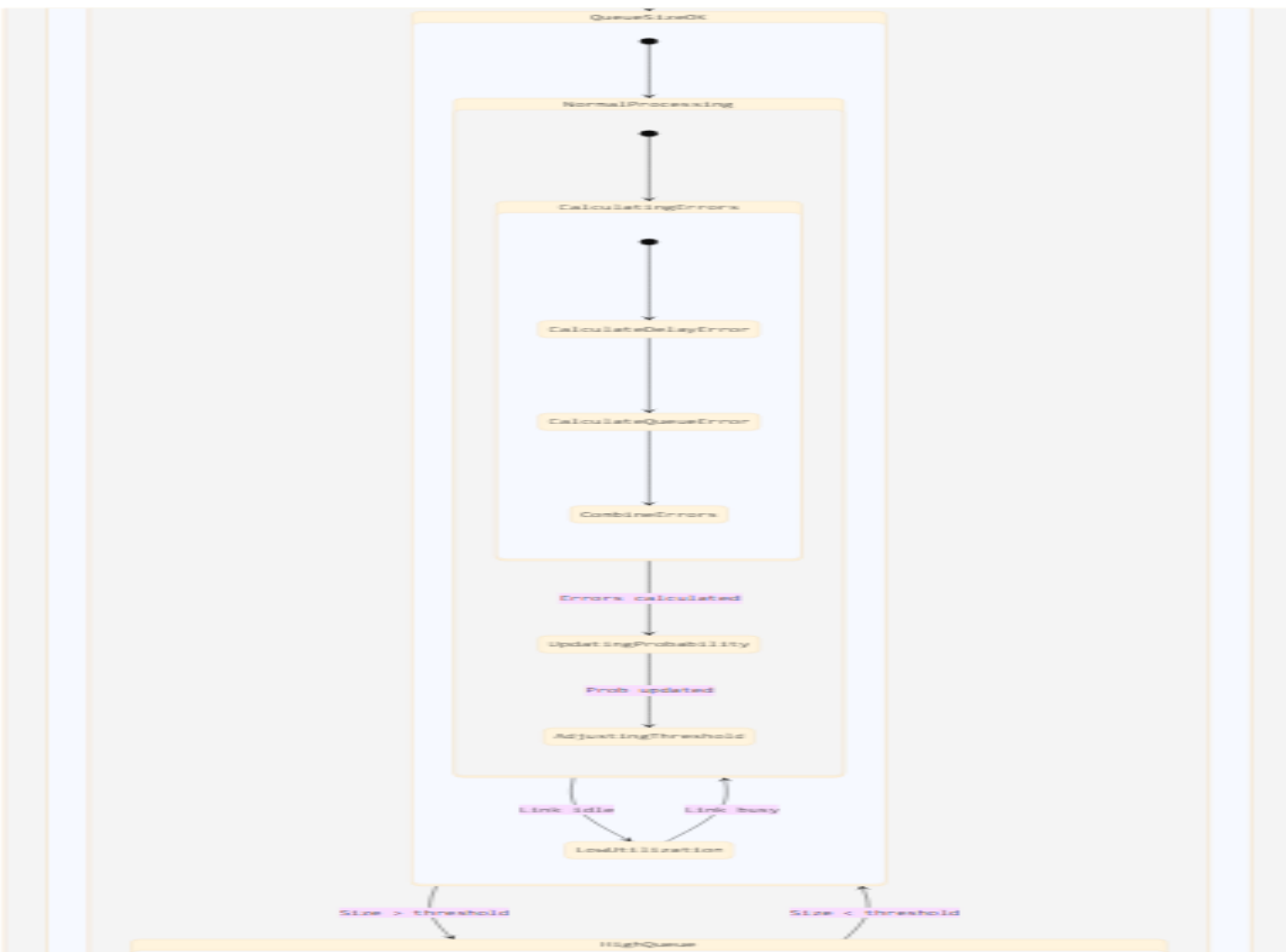
```
current_delay = current_time - packet_arrival_time
```

2.Error Signal Computation:

```
1 delay_error = current_delay - target_delay
2 queue_error = (current_queue_size - previous_queue_size) / capacity
3 total_error = delay_error + queue_error
```

3.Drop Probability Update:

```
drop_probability += ( $\alpha$  * error +  $\beta$  * accumulated_error)
```

3.Implementation Details

1.Packet Class:

Responsible of loading and generation packets with it's details:

```
@dataclass
class Packet:
    """Represents a network packet with all its properties."""
    packet_id: int
    data_length: int
    creation_time: float
    arrival_time: float = 0
    start_processing_time: float = 0
    completion_time: float = 0
    delete_time: float = 0
    drop_probability: float = 0 # PIE drop probability
```

3.Implementation Details

2.Event Logger:

Handles logging of simulation events:

```
class EventLogger:
    """Handles logging of simulation events with thread-safe operations

    > def __init__(self, start_time: float): ...
    > def _initialize_log_file(self) -> None: ...
    > def _get_elapsed_time(self) -> str: ...
    > def log_event(self, event: str) -> None: ...
```

```
def log_event(self, event: str) -> None:
    """Log an event with timestamp to both console and file."""
    with self.lock:
        timestamp = self._get_elapsed_time()
        log_message = f"{timestamp} - {event}"
        print(log_message)
        with open(self.events_file, 'a') as f:
            f.write(f"{log_message}\n")
```

3.Implementation Details

3. NetworkLink Class:

Represents a network link with transmission capabilities.:

```
class NetworkLink:
    """Represents a network link with transmission capabilities."""

    def __init__(self, speed: int, latency: float = 0.1): ...

    def transmit_packet(self, packet: Packet, sim_start_time: float) -> float:
        """Transmit a packet through the network link."""
        with self.lock:
            transmission_time = (packet.data_length / self.speed) + self.latency
            time.sleep(transmission_time)
            packet.arrival_time = time.time() - sim_start_time
            return transmission_time
```

3.Implementation Details

4. NetworkLink Class:

Represents a network link with transmission capabilities.:

```
class NetworkLink:
    """Represents a network link with transmission capabilities."""

    def __init__(self, speed: int, latency: float = 0.1): ...

    def transmit_packet(self, packet: Packet, sim_start_time: float) -> float:
        """Transmit a packet through the network link."""
        with self.lock:
            transmission_time = (packet.data_length / self.speed) + self.latency
            time.sleep(transmission_time)
            packet.arrival_time = time.time() - sim_start_time
            return transmission_time
```

3.Implementation Details

5. StatisticsCollector Class:

Collects and plots simulation statistics over time:

```
class StatisticsCollector:
    """Collects and plots simulation statistics over time."""

    def __init__(self): ...

    def record_statistics(self, current_time: float, queue: 'PacketQueue') -> None: ...

    def plot_statistics(self) -> None: ...
```

3.Implementation Details

6. StatisticsCollector Class:

Collects and plots simulation statistics over time:

```
class StatisticsCollector:
    """Collects and plots simulation statistics over time."""

    def __init__(self): ...

    def record_statistics(self, current_time: float, queue: 'PacketQueue') -> None: ...

    def plot_statistics(self) -> None: ...
```

3.Implementation Details

7. Simulation Class:

Main simulation class that coordinates the entire process

```
class Simulation:
    """Main simulation class that coordinates the entire process."""

    def __init__(self, queue_capacity: int, network_speed: int, generation_speed: float = 0.05, csv_file: str = "packets.csv"):
        pass

    def _load_packets_from_csv(self) -> List[Dict[str, int]]: ...

    def _collect_statistics(self) -> None: ...

    def generate_packets(self) -> None: ...

    def process_packets(self) -> None: ...

    def _print_statistics(self) -> None: ...

    def _calculate_avg_processing_time(self) -> float: ...

    def run(self) -> None: ...
```

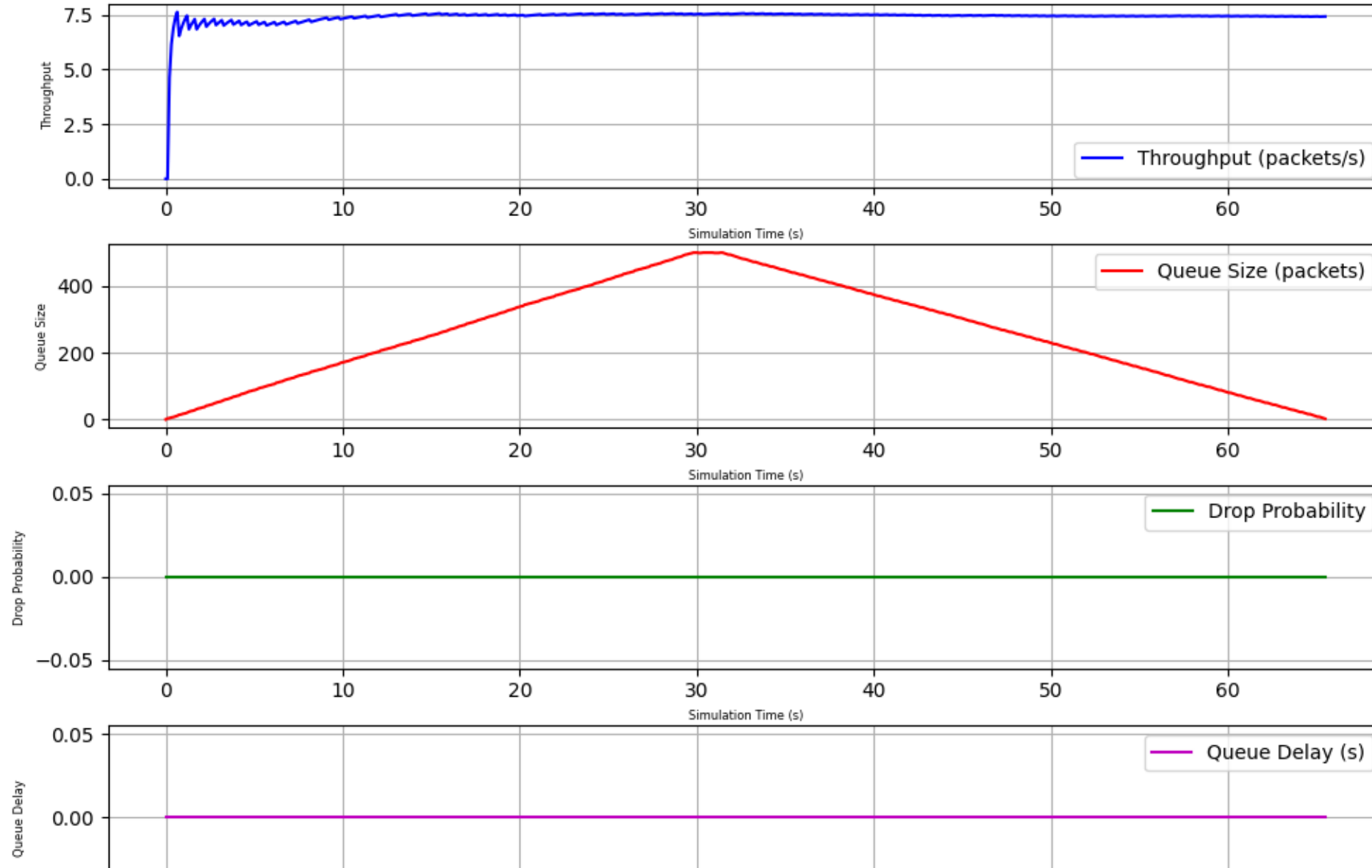

3.Implementation Details

0. Main Method:

Start point of the project

```
def main():  
    """Main entry point of the simulation."""  
    simulation = Simulation(  
        queue_capacity=500, # queue capacity (packets)  
        network_speed=100000, # Increased to 100mbps  
        generation_speed=0.05, # 50ms between packets  
        csv_file="packets.csv"  
    )  
    simulation.run()  
  
if __name__ == "__main__":  
    main()
```

4. Results and Analysis

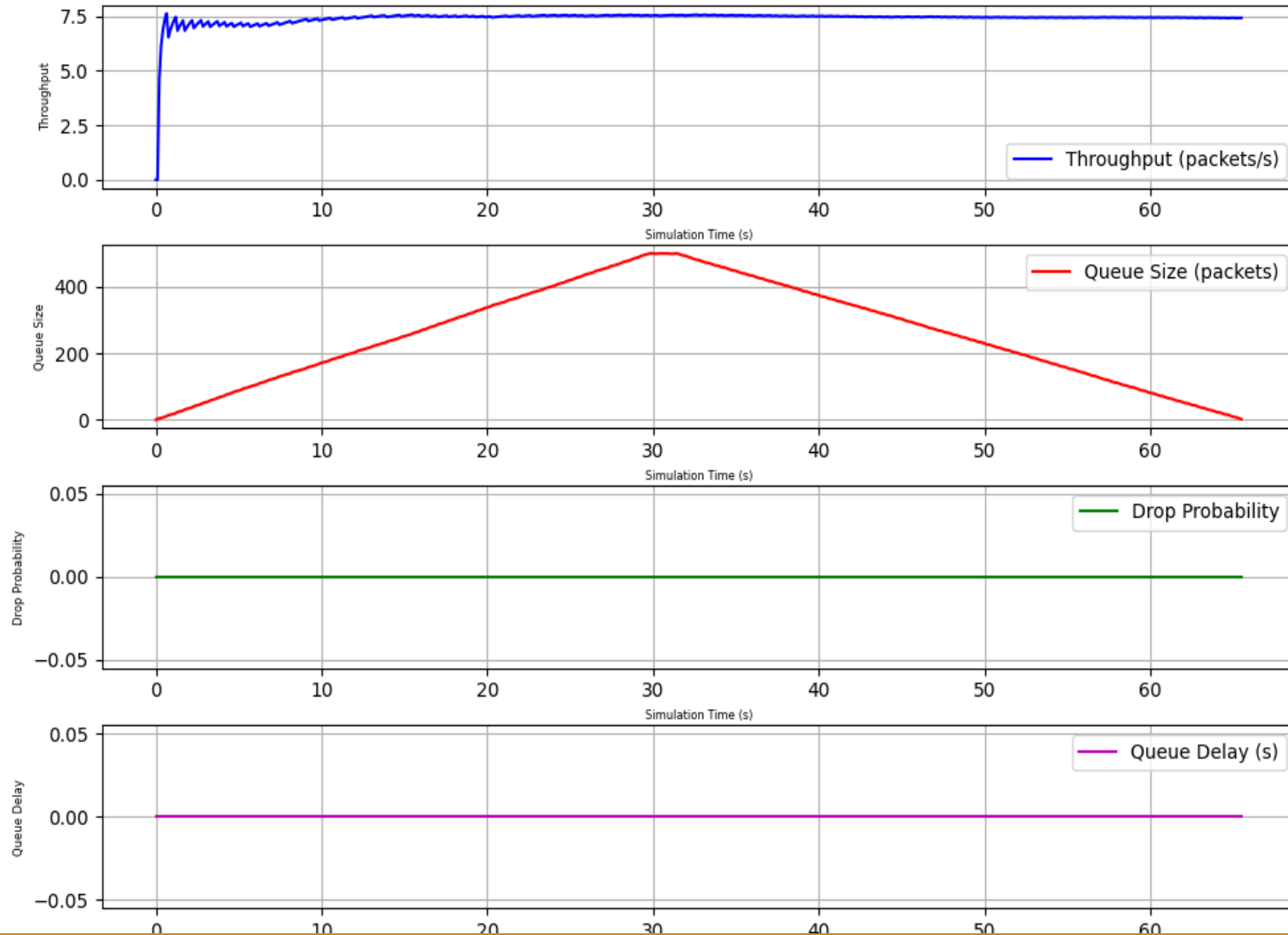


4. Results and Analysis

FIFO ALGORITHM

1. WEBSITE(HTTP) dataset
2. 1000 packets

```
=== Simulation Statistics ===  
Total Simulation Time: 96.62s  
Total Packets Generated: 1000  
Total Packets Processed: 970  
Total Packets Dropped: 30  
Average Processing Time: 0.00s  
Queue Capacity: 500  
=====  
  
01:36.619 - === All packets processed - Exiting ===
```



4. Results and Analysis

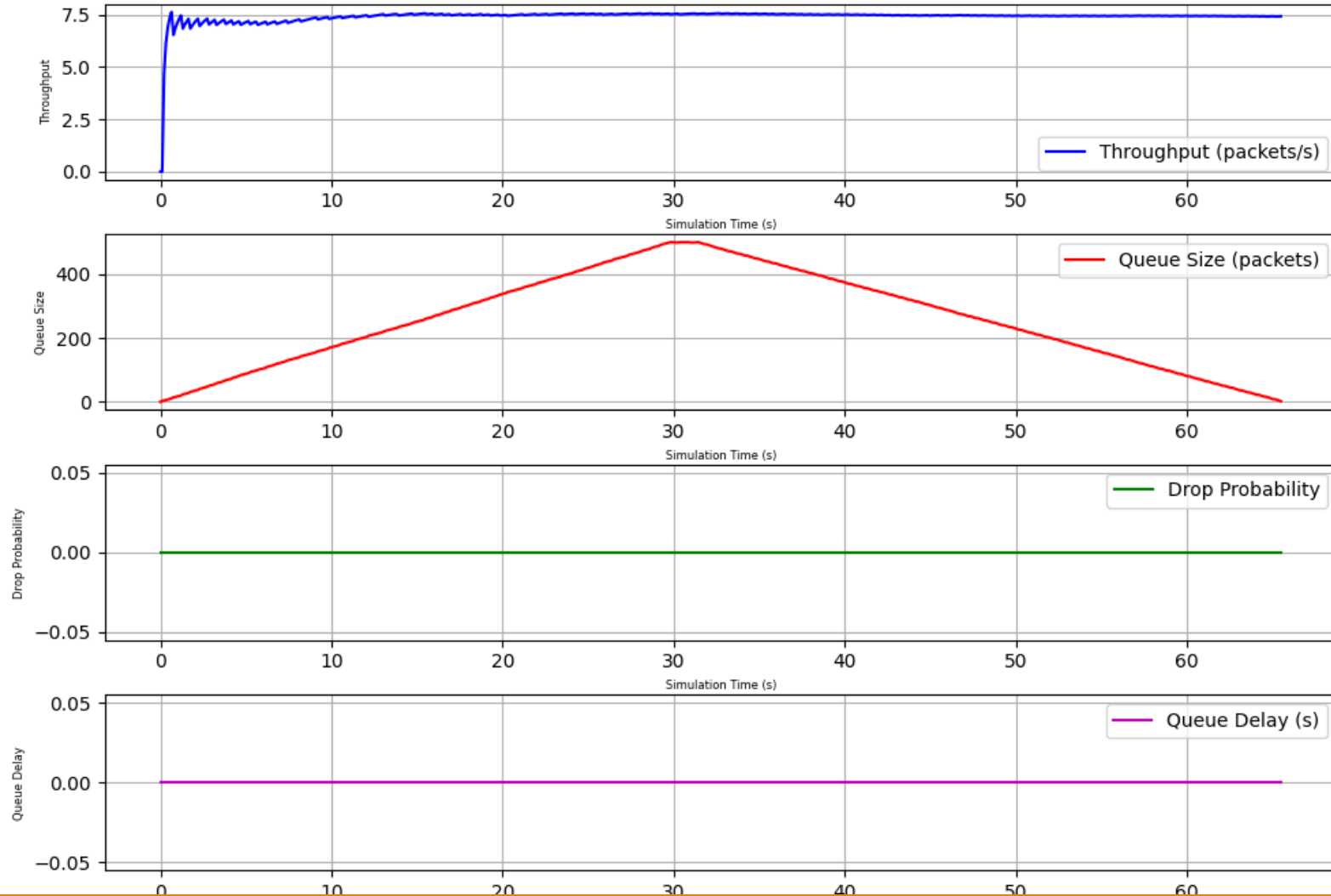
PIE ALGORITHM

1. WEBSITE(HTTP) dataset
2. 1000 packets

```
=== Simulation Statistics ===  
Total Simulation Time: 65.60s  
Total Packets Generated: 1000  
Total Packets Processed: 974  
Total Packets Dropped: 26  
Average Processing Time: 0.00s  
Average Queue Delay: 0.00s  
Queue Capacity: 500
```

```
=====
```

```
01:05.607 - === All packets processed - Exiting ===
```



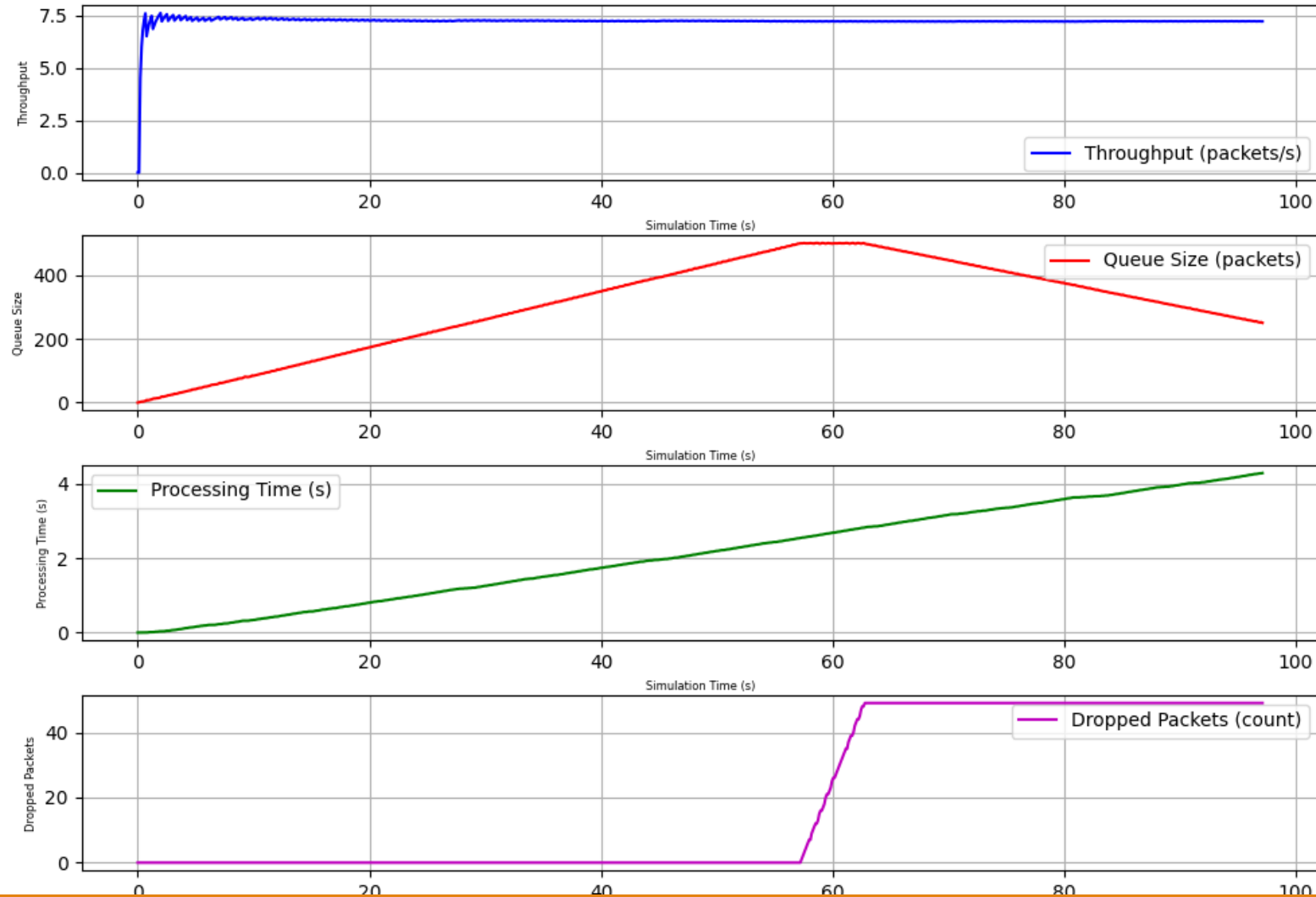
4. Results and Analysis

FIFO ALGORITHM

1. VIDEO dataset
2. 1000 packets

```
=== Simulation Statistics ===  
Total Simulation Time: 97.16s  
Total Packets Generated: 1000  
Total Packets Processed: 951  
Total Packets Dropped: 49  
Average Processing Time: 0.01s  
Queue Capacity: 500
```

```
01:37.161 - === All packets processed - Exiting ===
```



4. Results and Analysis

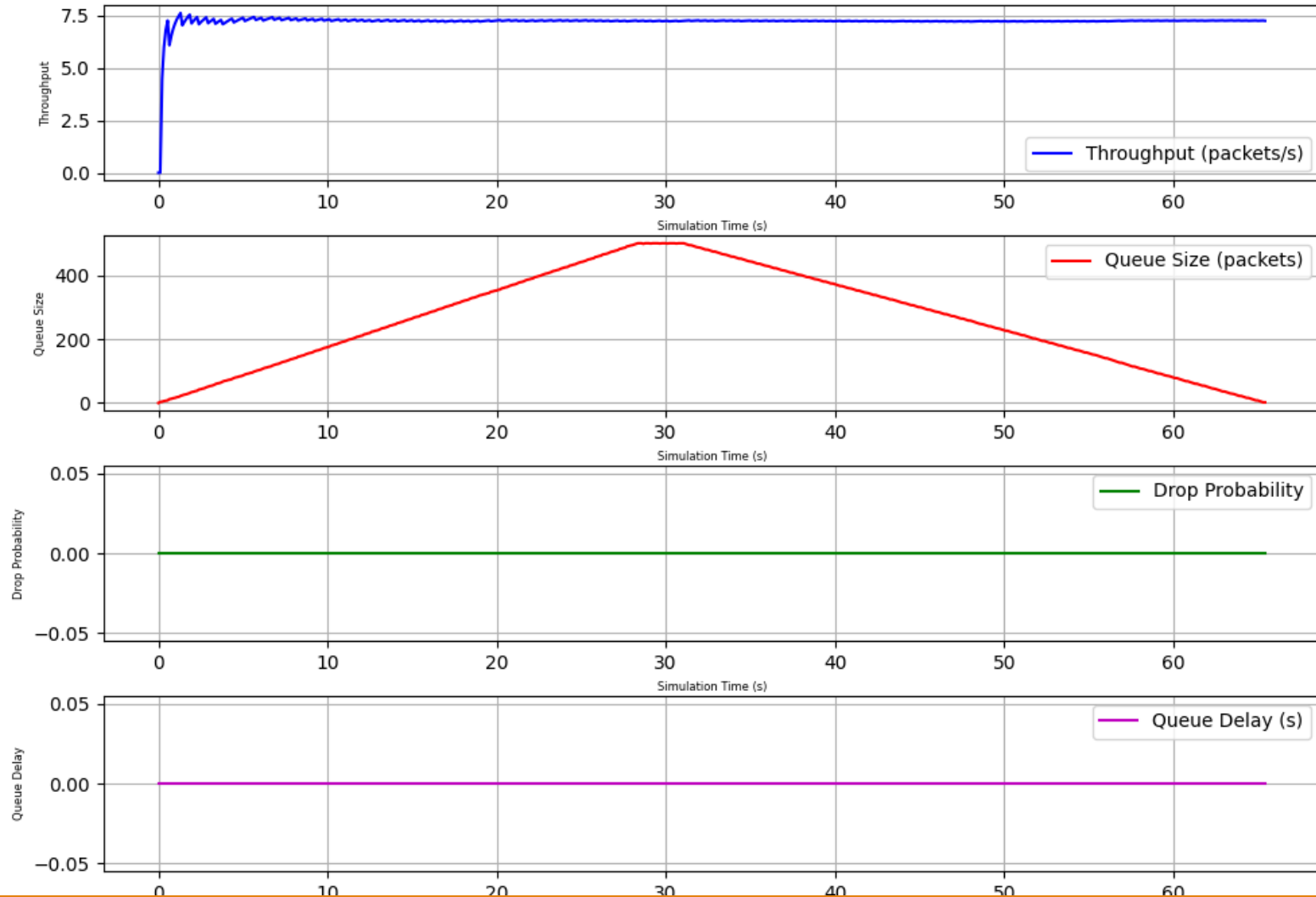
PIE ALGORITHM

1. VIDEO dataset
2. 1000 packets

```
=== Simulation Statistics ===  
Total Simulation Time: 65.43s  
Total Packets Generated: 995  
Total Packets Processed: 955  
Total Packets Dropped: 45  
Average Processing Time: 0.01s  
Average Queue Delay: 0.00s  
Queue Capacity: 500
```

```
=====
```

```
01:05.439 - === All packets processed - Exiting ===
```

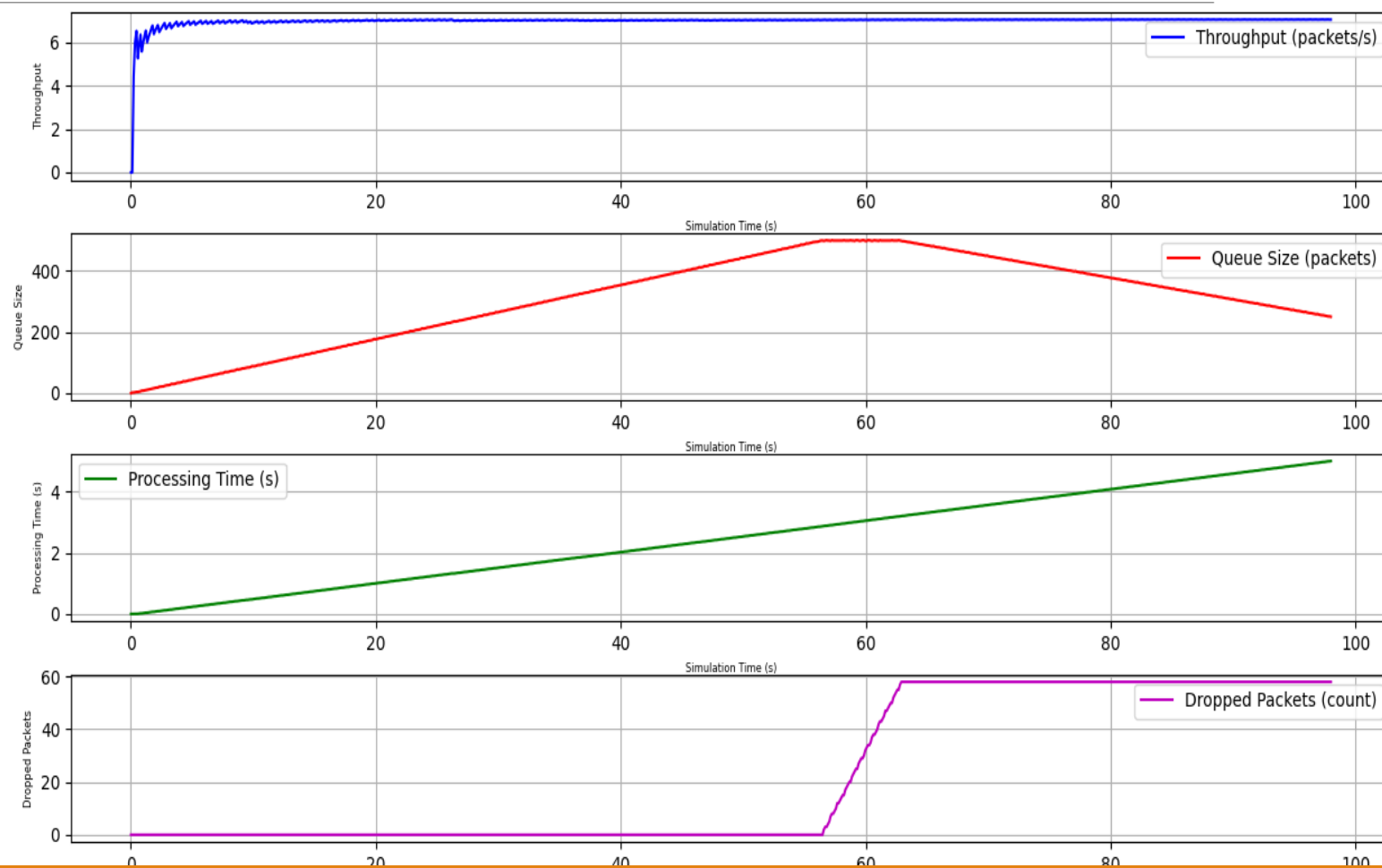


4. Results and Analysis

FIFO ALGORITHM

1. FTP dataset
2. 1000 packets

```
=== Simulation Statistics ===  
Total Simulation Time: 98.03s  
Total Packets Generated: 1000  
Total Packets Processed: 942  
Total Packets Dropped: 58  
Average Processing Time: 0.01s  
Queue Capacity: 500  
=====  
01:38.033 - === All packets processed - Exiting ===
```



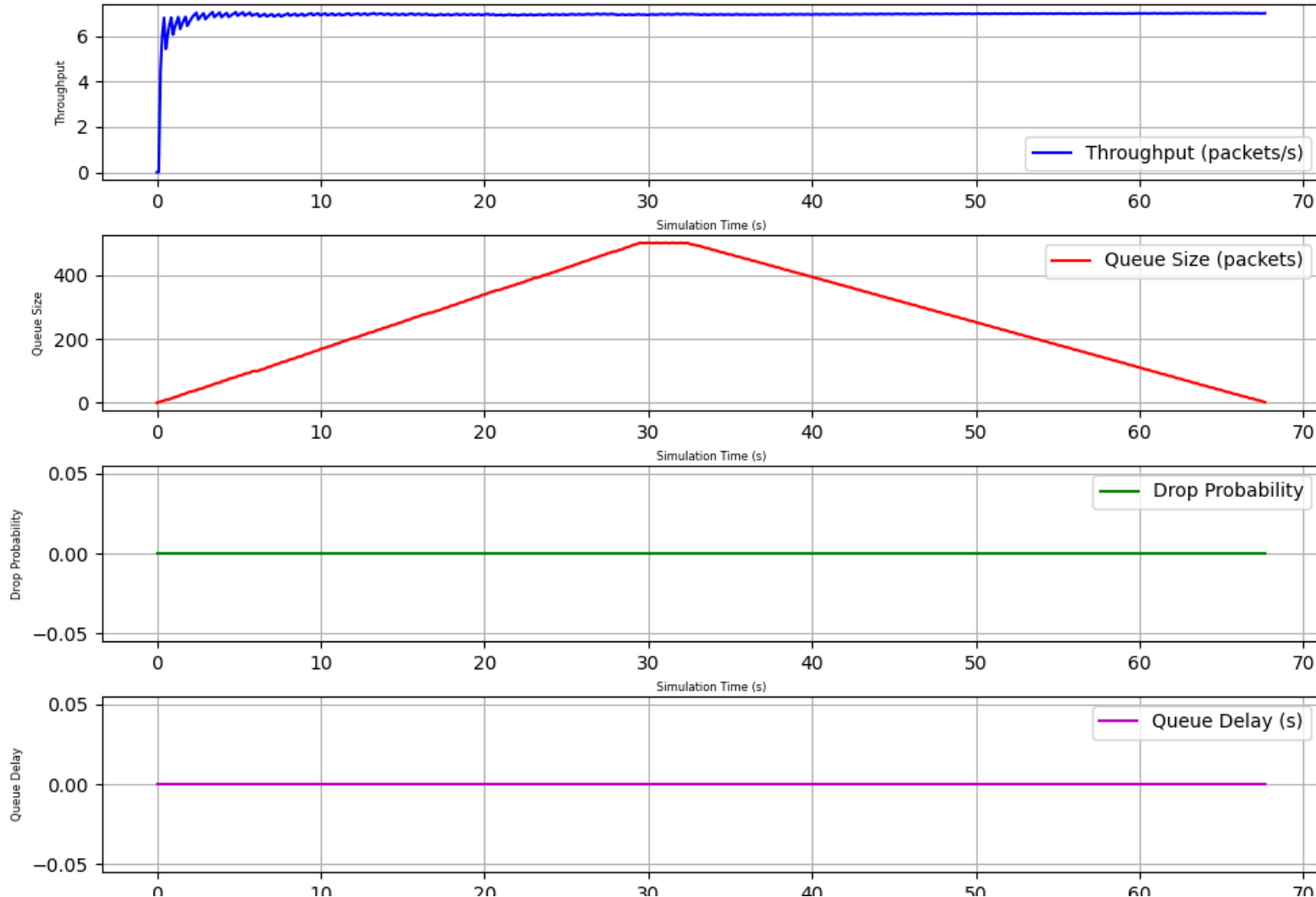
4. Results and Analysis

PIE ALGORITHM

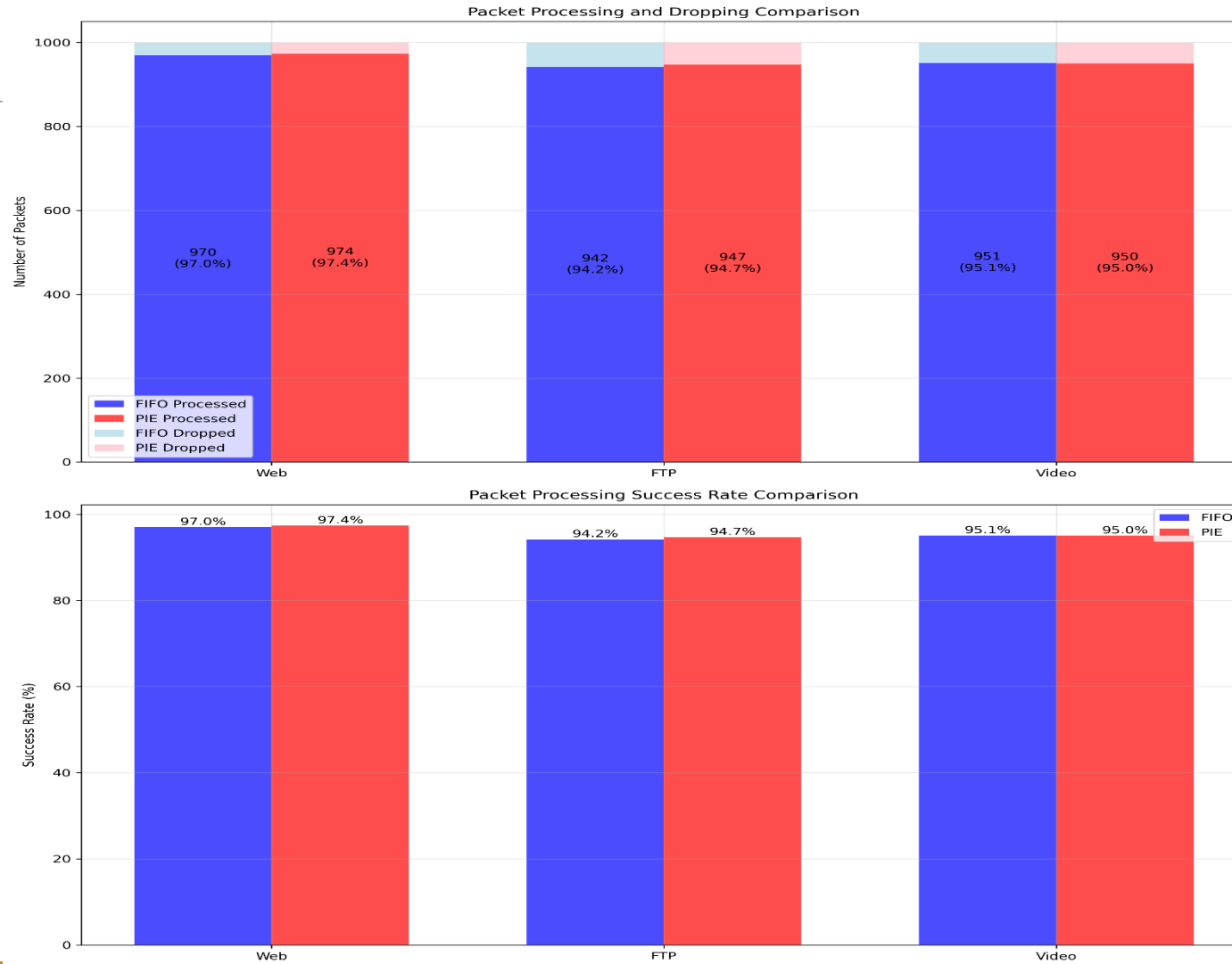
1. FTP dataset
2. 1000 packets

```
=== Simulation Statistics ===  
Total Simulation Time: 67.76s  
Total Packets Generated: 1000  
Total Packets Processed: 475  
Total Packets Dropped: 50  
Average Processing Time: 0.01s  
Average Queue Delay: 0.00s  
Queue Capacity: 500  
=====
```

01:07.758 - === All packets processed - Exiting ===



4. Results and Analysis



5. Results Discussion

Key Advantages of PIE:

- Higher success rates for web and FTP
- 30% faster processing time
- Better overall performance
- More efficient queue management

6. Future Improvements

Several enhancements can be made to the simulation framework:

1. Network Topology:

Multi-node network simulations

Complex routing scenarios

Network congestion modeling.

2. Visualization Enhancements:

Real-time monitoring dashboard

Interactive parameter tuning

Advanced statistical analysis tools

Source Code



Thank you

